# Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

## 5 - Rails (part 1)

SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Proff. Armando Fox and Roberto Beraldi

DIAG
Dipartimento di Ingegneria informatica, automatica e gestionale Antonio Ruberti

# Rails

Rails is a web application development framework written in Ruby

https://guides.rubyonrails.org/getting_started.html

Rails is designed to make programming web applications easier by making assumptions about what every developer needs to get started (i.e., “how to do more with less” if compared to many other languages and frameworks).

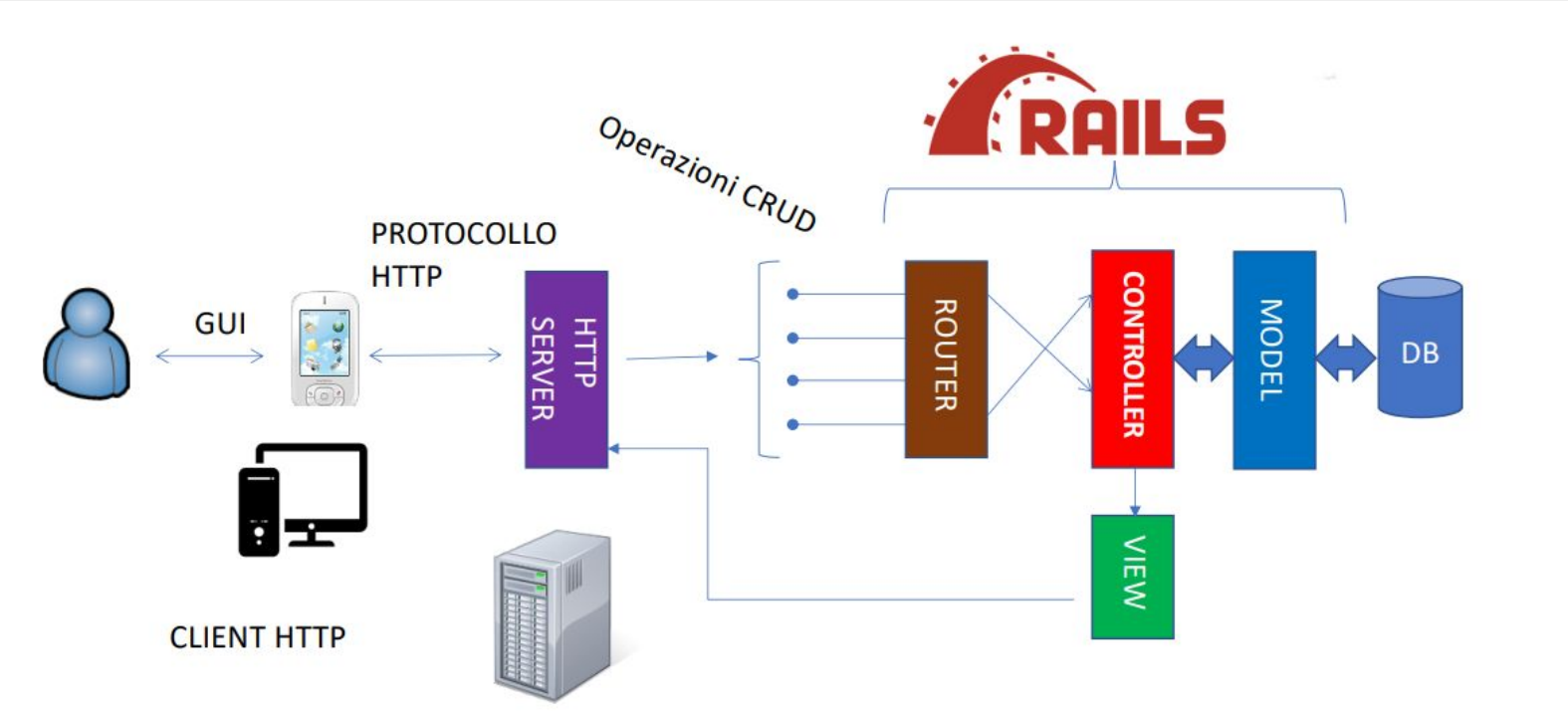
Rails is **opinionated** software: it assumes there is a "best" way to do things and is designed to encourage that way (and, in some cases, to discourage alternatives)

# Rails

Rails is designed to support apps that follow the **Model-View-Controller** pattern

The Rails philosophy includes two major guiding principles:

- **Don't Repeat Yourself:** a principle of software development that states that "Every piece of knowledge must have a single, unambiguous, authoritative representation within a system".
- **Convention Over Configuration**: Rails has opinions about the best way to do many things in a web application and defaults to this set of conventions (i.e., you don’t have to specify minimal details through lengthy configuration files).




Rails is a **server-side** framework: it **maps HTTP routes to code** in the app that performs the corresponding action. Its view subsystem **generates HTML pages**, as it was originally designed for apps whose client was a Web browser.

# Creating a test Rails application

```
rails new testapp \ # add flags below for faster creation
--skip-test-unit --skip-webpack-install --skip-javascript

cd testapp

bin/rails server
```

Check out **BIAR.md** or [https://guides.rubyonrails.org/getting_started.html](https://guides.rubyonrails.org/getting_started.html)

biar@biar:~/Desktop/Lab-ASSI/testapp

```
biar@biar:~/Desktop/Lab-ASSI/testapp$ bin/rails server
=> Booting Puma
=> Rails 6.1.7.2 application starting in development
=> Run `bin/rails server --help` for more startup options
Puma starting in single mode...
* Puma version: 5.6.5 (ruby 2.7.2-p137) ("Birdie's Version")
*  Min threads: 5
*  Max threads: 5
*  Environment: development
*          PID: 31083
* Listening on http://127.0.0.1:3000
* Listening on http://[::1]:3000
Use Ctrl-C to stop
Started GET "/" for 127.0.0.1 at 2023-03-05 19:17:51 +0100
   (0.9ms)  SELECT sqlite_version(*)
Processing by Rails::WelcomeController#index as HTML
  Rendering /home/biar/.rvm/gems/ruby-2.7.2/gems/railties-6.1.7.2/lib/rails/templ
ates/rails/welcome/index.html.erb
  Rendered /home/biar/.rvm/gems/ruby-2.7.2/gems/railties-6.1.7.2/lib/rails/templa
tes/rails/welcome/index.html.erb (Duration: 5.4ms | Allocations: 443)
Completed 200 OK in 13ms (Views: 8.4ms | ActiveRecord: 0.0ms | Allocations: 3514)
```

localhost:3000

# Rails App Contents

| File/Folder | Purpose |
|---|---|
| app/ | Contains the controllers, models, views, helpers, mailers, channels, jobs, and assets for your application. You'll focus on this folder for the remainder of this guide. |
| bin/ | Contains the `rails` script that starts your app and can contain other scripts you use to set up, update, deploy, or run your application. |
| config/ | Contains configuration for your application's routes, database, and more. This is covered in more detail in Configuring Rails Applications. |
| config.ru | Rack configuration for Rack-based servers used to start the application. For more information about Rack, see the Rack website. |
| db/ | Contains your current database schema, as well as the database migrations. |

# Rails App Contents

| | |
|---|---|
| Gemfile<br>Gemfile.lock | These files allow you to specify what gem dependencies are needed for your Rails application. These files are used by the Bundler gem. For more information about Bundler, see the Bundler website. |
| lib/ | Extended modules for your application. |
| log/ | Application log files. |
| public/ | Contains static files and compiled assets. When your app is running, this directory will be exposed as-is. |
| Rakefile | This file locates and loads tasks that can be run from the command line. The task definitions are defined throughout the components of Rails. Rather than changing `Rakefile`, you should add your own tasks by adding files to the `lib/tasks` directory of your application. |
| README.md | This is a brief instruction manual for your application. You should edit this file to tell others what your application does, how to set it up, and so on. |

# Rails App Contents

| | |
|---|---|
| storage/ | Active Storage files for Disk Service. This is covered in Active Storage Overview. |
| test/ | Unit tests, fixtures, and other test apparatus. These are covered in Testing Rails Applications. |
| tmp/ | Temporary files (like cache and pid files). |
| vendor/ | A place for all third-party code. In a typical Rails application this includes vendored gems. |
| .gitattributes | This file defines metadata for specific paths in a git repository. This metadata can be used by git and other tools to enhance their behavior. See the gitattributes documentation for more information. |
| .gitignore | This file tells git which files (or patterns) it should ignore. See GitHub - Ignoring files for more information about ignoring files. |
| .ruby-version | This file contains the default Ruby version. |

# Model View Controller (MVC)

Recall that an MVC-organized application consists of:

1) **Models** (core functionality and data manipulated by the application)
   - Typically, **one model for** each type of **entity** manipulated
2) **Views** (display information to the user)
   - Contain information on the models (=> data) with which users can interact
3) **Controllers** (mediate interaction in both directions)
   - **One per model**
   - When user interacts with a view, a controller action is invoked for the activity
   - A controller can ask the model to modify/retrieve information and will update the user's view based on the result of the operation

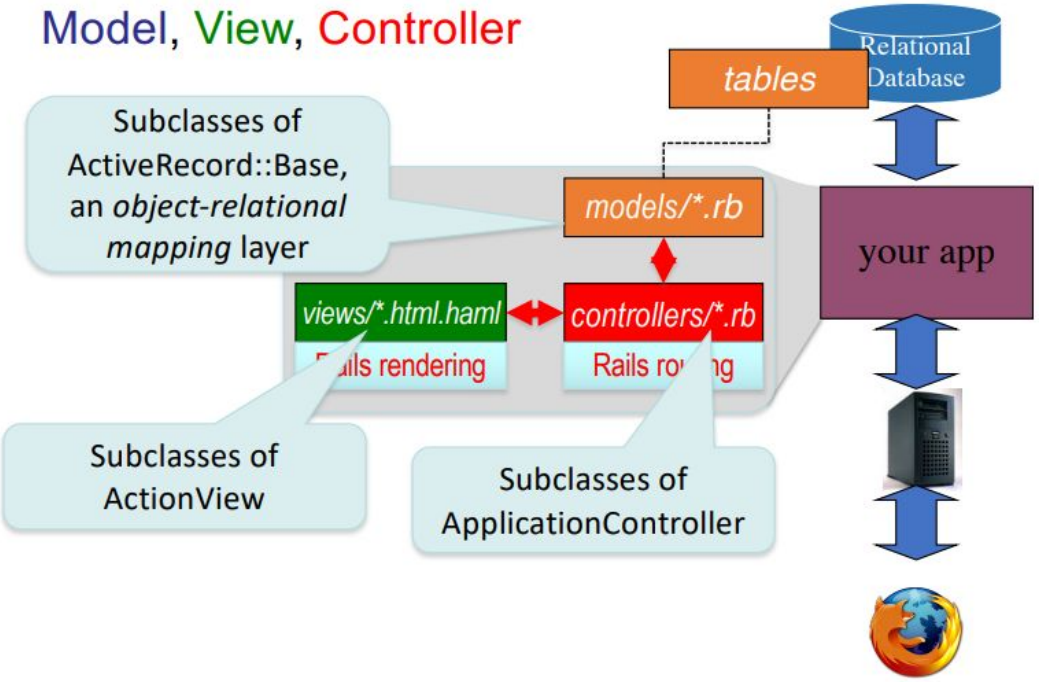
# Rails and MVC

Rails offers **base classes** from which your models, views, and controllers inherit.

Each **Rails model** is a resource type whose instances are rows in a particular type of relational database. Database-stored models are exposed to Ruby code via a design pattern known as **Active Record**.

A Rails app is a **collection of RESTful resources**: each has its own model, set of views, and controller. Resources may have **relationships** (e.g., a Movie can have many Reviews): these are captured by **foreign keys** in the database tables.

# Model, View, Controller



# Rails and RDBMS operation

Application needs to store and manipulate persistent data. Basic needs:

(1) Store different types of data items, where all instances of a particular type of **entity** share a common set of **attributes** (each instance can use a different value for a particular attribute)
(2) Express **relationships between** different kinds of **entities**.

These requirements are well known and **Relational DBMS (RDBMS)** provides a natural formalism to represent such structures and relationships. As we said, in Rails, **data** takes the form of a set of **resources stored in a relational database**.

But for (1), what is the correspondence between a resource stored in the DB and how Ruby sees it? And for (2), how are those two representations mediated?

# Active Record

Rails implements the **Active Record** architectural pattern.

**A Rails model is a class backed by a specific table of an RDBMS.** Hence, an instance of the class corresponds to a single row in that table.

Active Record gives each model the knowledge of how to **create**, **read**, **update**, and **delete** instances of itself in the database (**CRUD**).

The combination of table name and ID uniquely identifies a model instance.

# Active Record

The model has built-in behaviors that directly operate on the database for the four **CRUD** operations we just discussed: for example, to create a new row in the table to represent a new object.

All of the logic for “talking to” the database and for how to marshal/unmarhsal (that is, to (de)serialize) its attributes is implicitly included in each model.

Defining a class that inherits from **`ActiveRecord::Base`** provides all the necessary machinery to “connect” the model to the database. The directory app/models should contain one Ruby code file per model following specific naming conventions. Similar conventions apply to DB table names, attributes and their types, and constructor methods (see Chapter 4.2 of the book).

# Active Record and Migrations

A **migration** is a Ruby script for creating a database scheme

Migrations keep in sync the three different environments (each with distinct DB contents) for a Rails app: development, production, test

File `config/database.yml` specifies how to connect to the DB (default: sqlite)

Rails can create a Ruby file containing a migration class that descends from `ActiveRecord::Migration` and has a **`change`** instance method that you can fill in with the desired schema changes

See also: https://guides.rubyonrails.org/active_record_migrations.html#migration-overview

# Active Record and Migrations

Create a file `movie.rb` containing a `class Movie` as a subclass of `ActiveRecord::Base`

https://github.com/dcdelia/lab-assi/blob/2023-24/rails-intro/activerecord/movie_1.rb

movie_1.rb

```ruby
class Movie < ActiveRecord::Base
end
```

```
bin/rails generate migration CreateMovies
```

```
bin/rake db:migrate
```

# Active Record and Migrations

https://github.com/dcdelia/lab-assi/blob/2023-24/rails-intro/activerecord/initial_migration.rb

**initial_migration.rb**

```ruby
# replace 6.1 with your Rails version if you are not
# using the RVM 2.7.2 Ruby bundle on the BIAR VM
class CreateMovies < ActiveRecord::Migration[6.1]
  def change
    create_table 'movies' do |t|
      t.string 'title'
      t.string 'rating'
      t.text 'description'
      t.datetime 'release_date'
      # Add fields that let Rails automatically keep track
      # of when movies are added or modified:
      t.timestamps
    end
  end
end
```

# Active Record

```
bin/rails console

m = Movie.new

m.title = 'Mio Titolo'

m.title

m.save!

movie = Movie.where(:id => i).first
```

We can get seeds to initialize the DB here:
https://github.com/dcdelia/lab-assi/blob/2023-24/rails-intro/activerecord/seeds.rb

# All in one command

```
bin/rails generate model MovieV2 title:string rating:string
description:text release_date:datetime
```

# Controller and View

The **controller** can ask the model to retrieve or modify information and, based on the outcome, decides what view (and related info) to present to the user.

With only `Movies` as model, there will be one controller: the `Movies` controller. The actions defined in that controller can handle each type of user interaction with any `Movie` view (e.g., clicking on links/buttons) and contain the necessary logic to obtain model data to render any of the `Movie` views.

While the MVC definition of View is rather broad, a **Rails view** is a special case called a **template** (or template view), in which static markup interspersed with variable substitution is processed by a generic logic engine.

# Default Routes and CRUDI

File **`config/routes.rb`** determines which code in the app should be invoked to handle an incoming request in the form of an HTTP route. There are defaults:

```
get    '/movies'          => 'movies#index',  :as => 'movies'
get    '/movies/:id/new'  => 'movies#new',    :as => 'new_movie'
post   '/movies'          => 'movies#create', :as => 'movie'
get    '/movies/:id'      => 'movies#show',   :as => 'movie'
get    '/movies/:id/edit' => 'movies#edit',   :as => 'edit_movie'
put    '/movies/:id'      => 'movies#update', :as => 'movie'
delete '/movies/:id'      => 'movies#destroy',:as => 'movie'
```

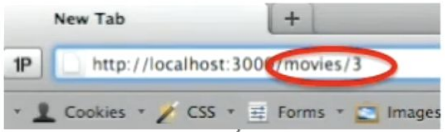
I
C
R
U
D

| Route | RESTful Action |
|---|---|
| GET /movies | I: List all movies (possibly with filtering) |
| POST /movies | C: Create new movie from attached form data |
| GET /movies/3 | R: Show info about movie whose ID=3 |
| PUT /movies/5 | U: Update movie ID 5 from attached form data |
| DELETE /movies/5 | D: Delete movie whose ID=5 |

# Default Routes and CRUDI

Parameters from the route go into the hash **`params[]`** prepared for the controller

1. Routes (in routes.rb) map incoming URL's to *controller actions* and extract any optional *parameters*
2. Controller actions set *instance variables,* visible to *views*
   - Subdirs and filenames of views/ match controllers & action names
3. Controller action eventually *renders* a view
   - may be HTML page, JSON object, etc.




config/routes.rb

app/controllers/movies_controller.rb

```
def show
 id = params[:id]
 @m=Movie.find(id)
end
```

app/views/movies/show.html.haml

```
%li
 Rating:
 = @m.rating
```

```
get '/movies/:id' => 'movies#show'
```

# Hands-on

https://github.com/dcdelia/lab-assi/tree/2023-24/rails-intro/book-lab

HTML base docs: https://developer.mozilla.org/en-US/docs/Web/HTML/Element